DS5500 - Data Science Capstone
Iteration #04
Model Selection and Choices – Methodology

Jyothssena Gomatum Sreenivaasan, Sharan Giri

1 Purpose of the Methodology

CourseLens solves a specific failure mode of generic AI in education: the absence of course context. A student asking a well-known system like Claude a quick question before an exam may get a technically correct answer, but one that uses approaches the professor has not covered yet. This is the **context gap**, and it is the reason generic AI tools can create as much confusion as they resolve in a course setting.

The methodology behind CourseLens is built around three decisions that directly address this gap:

- **A. Closed, Verified Knowledge Base:** The system ingests only verified course materials, so every answer it produces is grounded in what the professor chose to teach.
- **B. Temporal Gated Retrieval:** Students can only query content up to their current lecture, preventing exposure to concepts not yet covered.
- **C. Intent Aware Generation:** The system classifies whether a student needs a concept explained or code debugged, and selects a prompt strategy accordingly.

Each of these decisions is architectural rather than instructional, meaning they are enforced at the system level and not dependent on prompt adherence.

2 Problem Statement

CourseLens is an LLM-based question answering system for educational course materials. The core task is a RAG system constrained by curriculum metadata, with an intent classification layer that routes queries to the appropriate generation strategy.

The system:

- Retrieves relevant content from a multi-document corpus
- Enforces a temporal boundary based on lecture progression
- Generates cited responses grounded strictly in retrieved material

2.1 Significance

- **Academic Gap:** Students frequently encounter comprehension blocks outside office hours. When TA support is unavailable before an exam, students either rely on generic AI tools (which ignore the syllabus) or give up entirely. CourseLens is designed to be a reliable, always-available teaching assistant that respects the academic context.
- **24x7 Availability:** Professors cannot personally answer every student question 24/7. A system that accurately answers course-specific questions and refuses to answer out-of-scope ones, reduces course staff burden without compromising academic integrity.
- **Personalized Guidance:** The two most critical gaps in current educational AI are the lack of personalized learning and the absence of human-like instructional interaction, both of which are top motivations in educational chatbot research. CourseLens addresses both through its Socratic guidance approach, rather than directly posting answers, the system gives hints, helping students reason through the problems and reinforce the concepts.

These features are designed to mirror how a good TA interacts with students, making the experience feel genuinely instructional rather than transactional. Beyond the student experience, interaction logs create a valuable feedback loop for academic staff. Patterns in what students ask, where they get stuck, and which concepts trigger the most confusion reveal where course material is unclear. These insights complement traditional office hours, giving academic staff a broader view of student understanding across the entire class.

3 Data Collection and Preparation

CourseLens ingests a closed set of verified course materials provided by the instructor from the following sources :

3.1 Sources

- **Class Lecture Slides (PPTX))**:
  Landscape-oriented PPTX files containing slide content, diagram and code examples from `https://www.cs.virginia.edu/c++programdesign/slides/`
- **Additional Lecture Notes (PDF)**:
  Portrait-Oriented PDFs elaborating on concepts introduced in slides. These are parsed separately from slideshows due to their structural differences `https://ocw.mit.edu/courses/6-096-introducti`

This guarantees the trustworthiness of the system because the information/ knowledge base used in building the system contains only instructor-verified content.

3.2 Data Description

The current corpus consists of C++ lecture materials across multiple lecture units, totaling several hundred slides and notes. Each ingested document is classified at parse time:

- `pdf_slideshow` : Detected via landscape orientation, processed by PDFSlideshowContentExtractor
- `pdf_notes` : Detected via portrait orientation, procedure by PDFNotesContentExtractor
- `pptx` : Processed directly by PPTXParser via python-pptx

3.3 Metadata Schema

Field	Type	Description
lecture_number	int	Curriculum position (1-indexed)
source_type	str	pdf_slideshow, pdf_notes, pptx
chunk_type	str	parent or child
slide_number	int	Slide/page index
source_file	str	Filename for citation
title	str	Section/slide title

3.4 Preprocessing Steps

- **PPTX Pipeline:** Shapes are extracted in precedence order (title → code → body). Bullet hierarchy is preserved using python-pptx paragraph indent levels. Duplicate content (same text appearing on consecutive "build" slides) is removed via a deduplication step that runs before parent-child swap to avoid misidentifying image-slide neighbors as multi-child evidence.
- **PDF Slideshow Pipeline:** PDFSlideshowContentExtractor uses PyMuPDF for text extraction. A critical fix applied here is `_reassemble_code_spans()` , which addresses token fragmentation; the code blocks in PDFs are often extracted as a stream of single characters or short fragments. This method re-joins them into syntactically coherent blocks before chunking.
- **PDF Notes Pipeline:** PDFNotesContentExtractor uses section number hierarchy (dot-counting in section labels such as 1., 1.1, 1.2.3) to identify section boundaries and assign heading levels, ensuring related content is chunked together for comprehensive, coherent chunks.
- **Parent-Child Chunking:** The parent-child architecture groups semantically related child chunks (individual slides) under a single parent chunk (a topic group capturing a conceptual unit). This enables retrieval at two granularities — child chunks for precise lookup and parent chunks for broader contextual explanation. During retrieval, if multiple children from

the same parent are retrieved, they are swapped for the parent to provide fuller context, which is particularly effective given the sparse nature of individual PPTX slides.

- **Embeddings:** All text content is embedded using `BAAI/bge-m3`, a locally-hosted model with an 8192-token context window. No API calls are made during embedding. The model runs entirely on local hardware, eliminating cost and latency dependencies.

4 Selection of Models

4.1 Model Considerations

- **Embedding Model:** Two options were evaluated: all-MiniLM-L6-v2, which is fast and lightweight with a 512-token limit, and BAAI/bge-m3, which has a larger context window and stronger performance on technical content. The 512-token limit of MiniLM disqualifies it for parent chunks, which regularly exceed that length. bge-m3 was selected.

- **Retrieval Strategy:** Dense vector search handles semantic similarity well. A query about copying memory will retrieve content about memcpy even without exact term overlap. This forms the core retrieval mechanism, combined with metadata pre-filtering on `lecture_number` to enforce curriculum boundaries before any vector search executes.

- **Generation Model:** Gemini was selected for its large context window, which allows the full retrieved context to be passed without truncation, and for its compatibility with LangChain via langchain-google-genai. The model is prompted with strict context grounding and is instructed to answer only from the retrieved chunks and to explicitly state it cannot find relevant context when the retrieved material does not support an answer.

- **Intent Classification:** Traditional ML classifiers were considered for the query routing decision. The challenge is that intent is semantic and not keyword-based. A student asking "why does my loop run forever" is asking a debugging question despite containing no explicit debug keywords. A lightweight LLM-based classifier via Gemini handles this more reliably than a trained classifier, avoids the need for labeled training data, and allows faster iteration given the project timeline. The QueryRouter is designed and currently being wired into the pipeline.

4.2 Final Model Selection

The current stack is BAAI/bge-m3 for embeddings, ChromaDB for vector storage, Gemini via LangChain for generation, and a custom retrieval service handling the parent-child swap logic and temporal filtering. The LangChain pipeline connects these components on the query side and the ingestion pipeline runs independently and populates ChromaDB offline.

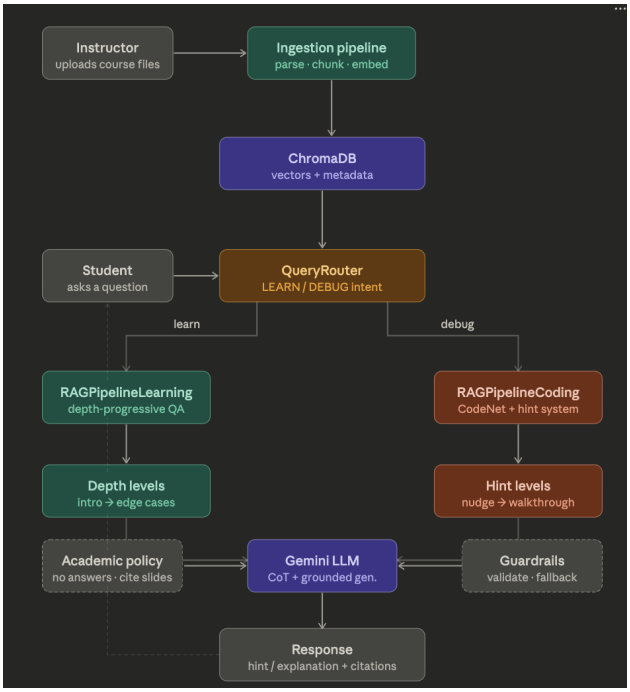

Figure 1: Enter Caption

5 Model Development and Training

5.1 Architecture

CourseLens does not involve model training. Both the embedding model and the LLM are used as pretrained components. The architecture of the system is the RAG pipeline design itself.

- **Ingestion Pipeline:** Raw files enter a file classifier that determines parser type. The appropriate parser extracts structured content. A content cleaner runs deduplication and code reassembly. The chunk builder constructs the parent-child hierarchy. The embedder encodes each chunk. ChromaDB stores vectors alongside metadata.
- **Retrieval Pipeline:** A student query arrives with the student's current lecture number. The retrieval service applies a lecture_number $lte metadata filter to ChromaDB before executing vector search. This means out-of-scope content is never fetched, never passed to Gemini, and never influences the response. Retrieved chunks go through the parent-child swap logic. Gemini generates a response citing the source file and slide number.
- **Progressive Disclosure:** Enforcing curriculum boundaries at the metadata level rather than the prompt level is a deliberate design choice. Prompt-level guardrails ask the model to ignore certain content, but if that content appears in the context window the model may still be influenced by it. The $lte filter prevents out-of-scope chunks from ever reaching the context window. If no in-scope chunks are relevant, the system returns an explicit refusal rather than falling back to model knowledge.

- **QueryRouter:** The QueryRouter is the first agentic component being introduced into the pipeline. It classifies student intent before retrieval begins, distinguishing between a conceptual question that needs a step-by-step explanation and a direct factual query. The DIRECT_QA path uses the existing retrieval pipeline. The CONCEPT_EXPLANATION path uses the same retrieval but applies a Socratic guidance prompt, instructing the system to ask questions and give hints rather than provide direct answers. This is currently being wired into the LangChain pipeline.

5.2 Training Process

No model training is performed. The embedding model and LLM are pretrained and used as-is. The RAG pipeline is configured rather than trained.

5.3 Hyperparameters

Retrieval parameters were set through qualitative evaluation on known query-answer pairs. The number of retrieved chunks before the parent-child swap is set to 5, which provides enough candidates to trigger the swap logic without flooding the context window. Chunk sizes target individual slides or note subsections for child chunks and 3–7 children for parent chunks, keeping both within the bge-m3 token limit while preserving coherent topic boundaries.

6 Evaluation Plan

Formal evaluation has not yet been conducted and is planned for the next project iteration once the full retrieval and generation pipeline is stable. This section defines the evaluation strategy that will be executed.

6.1 Planned Metrics

Three metrics are defined, each targeting a distinct component of the system.

- **Precision@k (Retrieval):** A golden set of approximately 25 query-source pairs will be manually constructed by writing natural student questions for known slide content and recording the expected source file and slide number. Precision@3 and Precision@5 will be computed, whether the correct chunk appears within the top 3 or top 5 retrieved results. This evaluates the retrieval layer independently of generation.
  ROUGE and WER are not used here. ROUGE measures n-gram overlap against a reference string, which is appropriate for summarization but poorly suited to question answering. A correct answer phrased differently from the reference will score low despite being accurate. Precision@k is the right tool for evaluating retrieval quality in this context.

- **Faithfulness score - generation quality:** For each query in the golden set, the generated response will be evaluated by an LLM judge against the retrieved context chunks. The

judge is prompted to determine whether the response contains any claim not supported by the retrieved context. Each response receives a binary score averaged across the set. This directly measures the core trust guarantee of CourseLens, that answers are grounded in course materials and do not hallucinate content.

- **Temporal gating accuracy - behavioral** A second set of approximately 15 queries will test progressive disclosure. Each query references a concept that first appears in a lecture beyond the student's current lecture number setting. The system should refuse to answer or return the out-of-scope response. Temporal gating accuracy is reported as the proportion of these cases handled correctly. This metric is specific to CourseLens and directly validates the curriculum-awareness guarantee that distinguishes it from a generic RAG deployment.

6.2 Summary Table

Metric	**Expected Set Size**	**Target**
Precision@3	25 queries	> 0.75
Precision@5	25 queries	> 0.85
Faithfulness Score	25 queries	> 0.85
Temporal Gating Accuracy	15 queries	1.00

7 Comparison with Related Work

CourseLens uses a more structured ingestion pipeline than typical RAG deployments, which treat all course text as a flat corpus. The parent-child hierarchy and unified metadata schema are not standard in off-the-shelf RAG implementations. Compared to GraphRAG (Jain et al.), CourseLens avoids graph construction overhead by encoding curriculum structure directly in metadata. This trades some cross-concept relationship inference for significantly lower resource cost and simpler deployment. Compared to the generic chatbot deployments surveyed by Ma et al., CourseLens enforces its context boundary at the vector store query level rather than relying on prompt instructions, making the temporal constraint substantially more robust.

8 References

1. Ma, W., Hu, Y. et al. (2025). *AI-based chatbots in higher education.* Education and Information Technologies.

2. Jain, A., Cui, L., & Chen, S. *Aligning LLMs for the Classroom with Knowledge-Based Retrieval.*